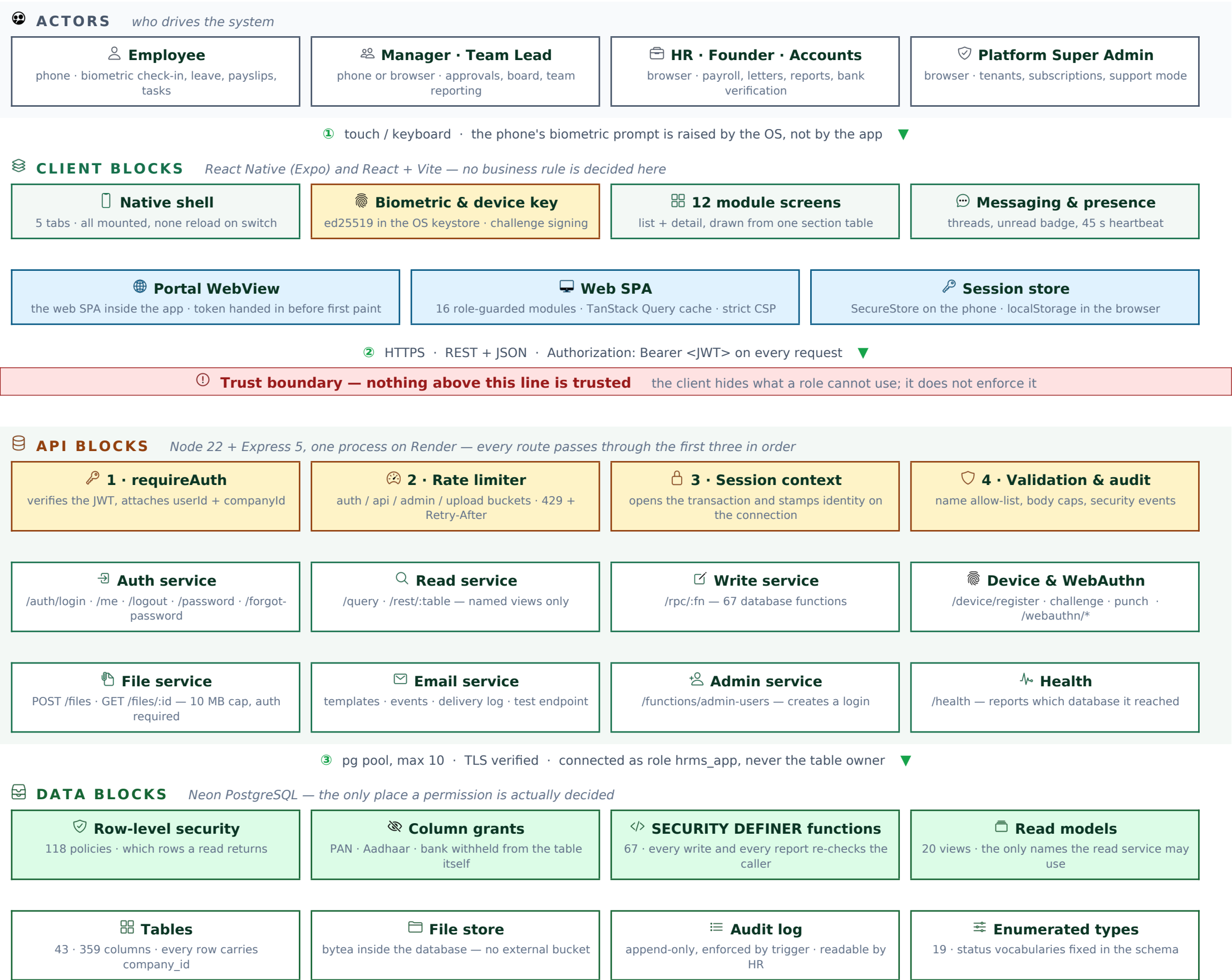






Interface definitions

every numbered connection on the diagram, and what actually crosses it

#	Interface	What crosses it
①	Actor → Client	Touch, keyboard, camera, GPS. The biometric prompt is raised by the operating system when the app reads its private key — the app cannot skip or fake it.
②	Client → API	HTTPS, REST, JSON. Authorization: Bearer <JWT>, 8 h TTL. Body capped at 100 KB, or 10 MB on a file or punch route. The mobile app and the web SPA use the same endpoints; there is no mobile backend.
③	API → Database	node-postgres pool, max 10, TLS verified. Each request opens a transaction and sets userId and companyId on that connection before any statement — this is what makes row-level security work on Neon, and it is cleared before the connection returns to the pool.
④	API → SMTP	Nodemailer over the tenant's own SMTP account. Credentials are encrypted with a key held outside the database, so a database dump contains no usable credential. Sending is disabled entirely unless the mode is production.
⑤	SMTP → Inbox	Outbound only. The HRMS receives no mail. Every send is written to the delivery log whether it succeeded or not.
⑥	Repository → Expo EAS	Builds the Android APK and iOS app. The built app points at the API's permanent address, not at a developer machine.
⑦	Repository → Vercel	Static build of the frontend. Security headers and the /api rewrite come from vercel.json; the rewrite is why the browser never talks to the API host directly.
⑧	Repository → Render	Node 22 build of server/. Health check at /health. DATABASE_URL, JWT_SECRET and the WebAuthn origin are stored by Render and are deliberately absent from the repository.
⑨	API ↔ Neon	The application role hrms_app. Never the owner: Postgres exempts owners from row-level security, and connecting as one would silently disable every tenant policy in the system.



Where each block runs

Block group	Runs on
Actors	Their own phone or browser
Native shell, biometric, module screens, messaging	The employee's handset — Android or iOS
Web SPA and the Portal WebView	Vercel, served as static files
All API blocks	One Node process on Render
All data blocks	Neon PostgreSQL
Email delivery	The tenant's own SMTP provider



The four enforcement blocks

in the order a request meets them

Order	Block	What it refuses
1	Navigation guard (client)	Nothing. It hides what a role cannot use so nobody is offered an action that will fail — courtesy, not security.
2	requireAuth (API)	A request with no valid session. Returns 401, and the client signs out cleanly.
3	Row-level security (database)	Rows belonging to another tenant, or to another employee where the role does not reach. Same query, different rows.
4	SECURITY DEFINER function (database)	The operation itself — an employee deciding their own leave, a manager reading salary, a payroll run by someone who may not.